

# Resolução de Equações Polinomiais

## Aula 3 — Cálculo Numéricos

Angela Leite Moreno

Cálculo Numérico para Ciência da Computação

March 18, 2026

$$p(x) = 0$$

- 1 Zeros de Funções Reais
- 2 Introdução e Motivação
- 3 Teoremas Fundamentais
- 4 Método da Bisseção
- 5 Método da Posição Falsa
- 6 Método do Ponto Fixo
- 7 Método de Newton-Raphson
- 8 Método da Secante
- 9 Métodos Específicos para Polinômios
- 10 Exercícios e Resumo

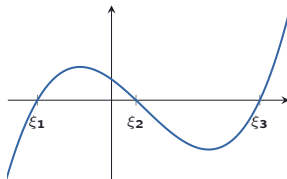
### O que é um zero (raiz) de uma função?

Dado uma função  $f(x)$ , chamamos de **zero** ou **raiz** de  $f$  qualquer número  $\xi$  tal que:

$$f(\xi) = 0.$$

Em nossos estudos  $\xi$  representará uma **raiz real**.

- ▶ Em problemas de Ciência e Engenharia é frequentemente necessário determinar  $\xi$  tal que  $f(\xi) = 0$ .
- ▶ As raízes podem ser reais ou complexas.
- ▶ Graficamente, as raízes reais são as **abscissas dos pontos de interseção** da curva  $y = f(x)$  com o eixo  $Ox$ .



O processo iterativo divide-se em duas fases:

## Fase I — Localização das raízes

Obter um intervalo  $[a, b]$  que contenha **uma única raiz**. Envolve análise gráfica e/ou teórica da função.

## Fase II — Refinamento

A partir de aproximações iniciais em  $[a, b]$ , melhorá-las sucessivamente até obter a raiz com precisão  $\varepsilon$  prefixada.

## Observações importantes

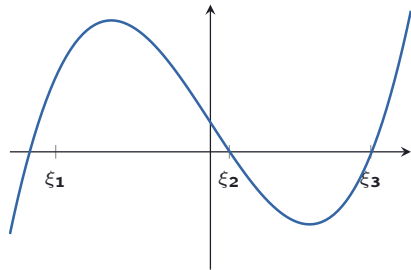
- ▶ A maioria dos métodos exige que a raiz  $\xi$  esteja isolada em  $[a, b]$ .
- ▶ Alguns métodos dispensam o isolamento prévio.
- ▶ É necessário definir **critérios de parada**.

**Método 1:** Esboçar o gráfico de  $f(x)$  e localizar as abscissas dos pontos de interseção com o eixo  $Ox$ .

### Exemplo

$$f(x) = x^3 - 9x + 3$$

$$\xi_1 \in [-3, -2], \quad \xi_2 \in [0, 1], \quad \xi_3 \in [2, 3]$$



**Método 2:** Reescrever  $f(x) = 0$  como  $g(x) = h(x)$  e localizar as interseções dos gráficos de  $g$  e  $h$ .

## Fase I — Análise Teórica (Teorema de Bolzano)

O teorema a seguir garante a existência de raiz em um intervalo:

### Theorem (Bolzano)

Se  $f$  é contínua em  $[a, b]$  e  $f(a)f(b) < 0$ , então existe pelo menos um ponto  $\xi \in (a, b)$  com  $f(\xi) = 0$ .

### Unicidade

Se adicionalmente  $f'(x)$  existir e preservar o sinal em  $[a, b]$ , então existe **uma única raiz** no intervalo.

### Atenção

Se  $f(a)f(b) > 0$ , pode existir ou não raiz em  $[a, b]$ .

### Aplicação — Tabela de sinais

$$f(x) = x^3 - 9x + 3:$$

| $x$    | -3 | -2 | 0 | 1 | 2 | 3 |
|--------|----|----|---|---|---|---|
| $f(x)$ | -  | +  | + | - | - | + |

Mudanças de sinal:  $[-3, -2]$ ,  $[0, 1]$ ,  $[2, 3]$ .

Durante as iterações, usamos critérios para decidir quando parar:

### Critérios práticos

1. **Resíduo:**  $|f(x_i)| < \varepsilon$ .
2. **Erro absoluto:**  $|x_i - x_{i-1}| < \varepsilon$ .
3. **Erro relativo:**  $\frac{|x_i - x_{i-1}|}{|x_i|} < \varepsilon$ .

### Cuidado!

Os dois primeiros critérios nem sempre equivalem. Por exemplo:

- ▶ Função quase horizontal:  $|f(x_i)|$  pequeno, mas  $x_i$  longe da raiz.
- ▶ Função muito íngreme:  $x_i$  próximo da raiz, mas  $|f(x_i)|$  ainda grande.

### Recomendação

Combinar os critérios 1 e 2 sempre que possível. Definir também um número máximo de iterações para garantir término.

Resolver  $p(x) = 0$ , com:

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0, \quad a_n \neq 0.$$

## Contexto Histórico

- ▶ **Antiguidade:** Babilônios resolvem quadráticas ( $\sim 2000$  a.C.).
- ▶ **Séc. XVI:** Cardano e Ferrari: cúbicas e quárticas.
- ▶ **1824:** Abel prova: **sem fórmula geral** para  $n \geq 5$ .
- ▶ **Séc. XX:** Métodos numéricos robustos.

## ! Teorema de Abel-Ruffini

Não existe fórmula geral usando as quatro operações e radicais para  $n \geq 5$ .

**Métodos numéricos são essenciais!**

## Curvas de Bézier Cúbicas

Base de toda computação gráfica vetorial:

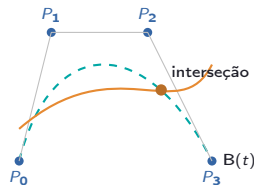
$$B(t) = (1-t)^3P_0 + 3(1-t)^2tP_1 + \dots + t^3P_3.$$

**Problema:** Interseção de duas curvas

$B_1(s) = B_2(t)$  leva a polinômio de grau até 9!

## Aplicações Práticas

- ▶ Fontes digitais: TrueType, PostScript.
- ▶ Design: Adobe Illustrator, Inkscape.
- ▶ Animação: Blender, Maya, After Effects.
- ▶ CAD: AutoCAD, SolidWorks.



Ref: Farin (2002).

## Teoria de Controle

Sistema linear:  $\frac{dx}{dt} = Ax$

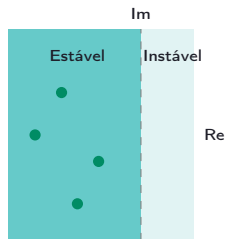
Estabilidade  $\Leftrightarrow$  raízes do **polinômio característico**:

$$p(\lambda) = \det(\lambda I - A) = 0$$

## Critério de Estabilidade

Sistema estável  $\Leftrightarrow$  todas as raízes satisfazem:

$$\operatorname{Re}(\lambda_i) < 0 \quad \forall i$$



Ref: Ogata (2010).

## Theorem (Gauss, 1799)

*Todo polinômio  $p(x)$  de grau  $n \geq 1$  com coeficientes em  $\mathbb{C}$  possui pelo menos uma raiz em  $\mathbb{C}$ .  
Equivalentemente:*

$$p(x) = a_n(x - r_1) \cdots (x - r_n), \quad r_i \in \mathbb{C}.$$

## Estratégia da demonstração

**Passo 1.** Mostrar que  $|p(z)| \rightarrow \infty$  quando  $|z| \rightarrow \infty$ .

**Passo 2.** Como  $|p|$  é coerciva e contínua, ela atinge seu mínimo global em algum  $z_0 \in \mathbb{C}$ .

**Passo 3.** Supor  $p(z_0) \neq 0$  e construir  $h$  tal que  $|p(z_0 + h)| < |p(z_0)|$  — contradição com a minimalidade. Logo  $p(z_0) = 0$ .

$|p(z)| \rightarrow \infty$  quando  $|z| \rightarrow \infty$

Fatore o coeficiente líder:

$$p(z) = a_n z^n \left( 1 + \frac{a_{n-1}}{a_n z} + \cdots + \frac{a_0}{a_n z^n} \right).$$

Para  $|z| > 0$ , divida por  $|z|^n$ :

$$\frac{|p(z)|}{|z|^n} \geq |a_n| - \frac{|a_{n-1}|}{|z|} - \cdots - \frac{|a_0|}{|z|^n} \xrightarrow{|z| \rightarrow \infty} |a_n| > 0.$$

Logo, dado  $M > 0$ , existe  $R > 0$  tal que  $|p(z)| > M$  para todo  $|z| > R$ .

## Consequência (Passo 2)

A função contínua  $|p|$  restrita ao disco fechado  $\overline{D}_R = \{|z| \leq R\}$  é contínua num compacto, logo atinge seu mínimo em algum  $z_0 \in \overline{D}_R$ . Como  $|p| > M$  fora de  $D_R$ , esse  $z_0$  é mínimo global.

Se  $p(z_0) \neq 0$ , há contradição

Escreva  $p(z_0) = \alpha \neq 0$  e expanda em série de Taylor em torno de  $z_0$ :

$$p(z_0 + h) = \alpha + c_m h^m + O(h^{m+1}), \quad c_m \neq 0, \quad m \geq 1,$$

em que  $m$  é o menor índice com coeficiente não nulo após  $\alpha$ .

Escolha a fase de  $h$ : tome  $h = t e^{i\theta}$  com  $\theta$  tal que  $c_m e^{im\theta} = -|c_m|$  e  $t > 0$  pequeno. Então:

$$p(z_0 + h) = \alpha - |c_m| t^m + O(t^{m+1}).$$

Tomando módulo e usando  $|\alpha - s| < |\alpha|$  para  $0 < s < 2|\alpha|$  real:

$$|p(z_0 + h)| = |\alpha| \left| 1 - \frac{|c_m|}{\alpha} t^m + O(t^{m+1}) \right| < |\alpha| = |p(z_0)|,$$

para  $t$  suficientemente pequeno. Isso contradiz  $z_0$  ser mínimo de  $|p|$ . Logo  $p(z_0) = 0$ . ■

## Corolário: Raízes Complexas Conjugadas

### Corolário (fatoração sobre $\mathbb{R}$ )

Um polinômio com coeficientes reais fatoriza como:

$$p(x) = a_n \prod_{i=1}^k (x - r_i) \prod_{j=1}^m (x^2 + b_j x + c_j), \quad b_j^2 - 4c_j < 0.$$

Os fatores quadráticos correspondem a pares de raízes complexas conjugadas.

### Demonstração

Se  $z = a + bi$  ( $b \neq 0$ ) é raiz de  $p \in \mathbb{R}[x]$ , tomamos o conjugado de  $p(z) = 0$ :

$$\overline{p(z)} = p(\bar{z}) = 0 \quad \Rightarrow \quad \bar{z} = a - bi \text{ também é raiz.}$$

**Por quê  $\overline{p(z)} = p(\bar{z})$ ?** Porque os coeficientes são reais:  $\overline{a_k z^k} = a_k \bar{z}^k$ .

O fator correspondente ao par  $z, \bar{z}$ :

$$(x - z)(x - \bar{z}) = x^2 - 2ax + (a^2 + b^2) \in \mathbb{R}[x].$$



## Corolário: Raízes Complexas Conjugadas

### Corolário (fatoração sobre $\mathbb{R}$ )

Um polinômio com coeficientes reais fatoriza como:

$$p(x) = a_n \prod_{i=1}^k (x - r_i) \prod_{j=1}^m (x^2 + b_j x + c_j), \quad b_j^2 - 4c_j < 0.$$

Os fatores quadráticos correspondem a pares de raízes complexas conjugadas.

### Demonstração

Se  $z = a + bi$  ( $b \neq 0$ ) é raiz de  $p \in \mathbb{R}[x]$ , tomamos o conjugado de  $p(z) = 0$ :

$$\overline{p(z)} = p(\bar{z}) = 0 \quad \Rightarrow \quad \bar{z} = a - bi \text{ também é raiz.}$$

**Por quê  $\overline{p(z)} = p(\bar{z})$ ?** Porque os coeficientes são reais:  $\overline{a_k z^k} = a_k \bar{z}^k$ . O fator correspondente ao par  $z, \bar{z}$ :

$$(x - z)(x - \bar{z}) = x^2 - 2ax + (a^2 + b^2) \in \mathbb{R}[x].$$



### Theorem (Teorema do Valor Intermediário — Bolzano)

Se  $f : [a, b] \rightarrow \mathbb{R}$  é contínua e  $f(a)f(b) < 0$ , então  $\exists c \in (a, b)$  com  $f(c) = 0$ .

### Demonstração via supremo

Suponha  $f(a) < 0 < f(b)$ . Defina  $S = \{x \in [a, b] : f(x) \leq 0\}$ .

$S$  é não vazio e limitado superiormente:  $a \in S$  e  $S \subseteq [a, b]$ .

Pelo **axioma do supremo**:  $c = \sup S \in [a, b]$  existe.

$f(c) \leq 0$ : Existe  $(x_n) \subset S$  com  $x_n \rightarrow c$ . Por continuidade:  $f(c) = \lim f(x_n) \leq 0$ .

$f(c) \geq 0$ : Como  $f(b) > 0$ , temos  $b \notin S$ , logo  $c < b$ . Existem  $y_n > c$  com  $y_n \rightarrow c$  e  $y_n \notin S$ , i.e.,  $f(y_n) > 0$ . Por continuidade:  $f(c) = \lim f(y_n) \geq 0$ .

Portanto  $f(c) = 0$ . ■

### Theorem (Cota de Cauchy)

Toda raiz de  $p(x) = a_n x^n + \cdots + a_0$  satisfaz  $|x| \leq 1 + M/|a_n|$ ,  $M = \max_{0 \leq i < n} |a_i|$ .

### Demonstração

**Caso  $|r| \leq 1$ :**  $|r| \leq 1 \leq 1 + M/|a_n|$ . ✓

**Caso  $|r| > 1$ :** De  $p(r) = 0$  extraímos  $a_n r^n = -(a_{n-1} r^{n-1} + \cdots + a_0)$ .

Tomando módulo e majorando:

$$|a_n| |r|^n \leq M(|r|^{n-1} + \cdots + 1).$$

Como  $|r| > 1$ , a soma geométrica satisfaz:

$$|r|^{n-1} + \cdots + 1 = \frac{|r|^n - 1}{|r| - 1} < \frac{|r|^n}{|r| - 1}.$$

Substituindo e cancelando  $|r|^n > 0$ :

$$|a_n|(|r| - 1) < M \quad \Rightarrow \quad |r| < 1 + \frac{M}{|a_n|}.$$



### Exemplo — Cota de Cauchy

Para  $p(x) = x^3 - 6x^2 + 11x - 6$ :

$$M = \max\{6, 11, 6\} = 11$$

$$|x| \leq 1 + \frac{11}{1} = 12$$

Todas as raízes estão no disco  $|x| \leq 12$ .

As raízes verdadeiras são 1, 2, 3 — bem menores que 12, mas a cota garante o intervalo de busca.

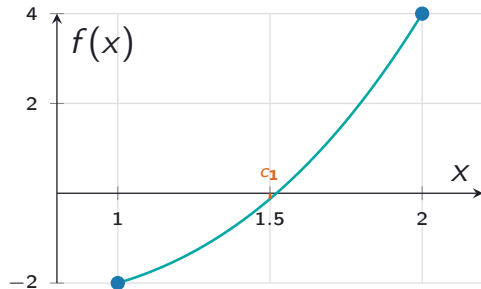
## Algoritmo

Dado  $[a, b]$  com  $f(a)f(b) < 0$ , calcular  $c = \frac{a+b}{2}$ :

- ▶  $f(c) = 0$ : raiz exata
- ▶  $f(a)f(c) < 0$ : raiz em  $[a, c]$ ,  $b \leftarrow c$
- ▶ caso contrário: raiz em  $[c, b]$ ,  $a \leftarrow c$

## Propriedades:

- Sempre converge.
- Iterações previsíveis.
- Sem derivada.



### Theorem (Convergência Linear da Bisseção)

Seja  $r$  a raiz exata em  $[a_0, b_0]$  e  $x^{(k)} = (a_k + b_k)/2$ . Então:

$$|x^{(k)} - r| \leq \frac{b_0 - a_0}{2^{k+1}}.$$

Para garantir  $|x^{(k)} - r| < \varepsilon$ : basta  $k \geq \log_2\left(\frac{b_0 - a_0}{\varepsilon}\right) - 1$ .

### Demonstração

**Comprimento do intervalo:** A cada iteração o intervalo é dividido ao meio:

$$b_k - a_k = \frac{b_{k-1} - a_{k-1}}{2} = \dots = \frac{b_0 - a_0}{2^k}.$$

### Demonstração – Continuação

**Erro:** Como  $r \in [a_k, b_k]$  e  $x^{(k)}$  é o ponto médio:

$$|x^{(k)} - r| \leq \frac{b_k - a_k}{2} = \frac{b_0 - a_0}{2^{k+1}}.$$

**Critério de parada:** Exige  $2^{k+1} > (b_0 - a_0)/\varepsilon$ , ou seja,  $k > \log_2\left(\frac{b_0 - a_0}{\varepsilon}\right) - 1$ . ■

### Exemplo:

$[0, 1]$ ,  $\varepsilon = 10^{-10}$ :  $k \geq \log_2(10^{10}) - 1 \approx 32$  iterações.

# Implementação da Bisseção

```
1 def bissecao(f, a, b, tol=1e-10, max_iter=100):
2     fa, fb = f(a), f(b)
3     if fa * fb > 0:
4         raise ValueError("f(a) e f(b) devem ter sinais opostos.")
5     historico = []
6     for k in range(max_iter):
7         c = (a + b) / 2
8         fc = f(c)
9         historico.append(c)
10        if abs(fc) < tol or abs(b - a) < tol:
11            return c, k+1, historico
12        if fa * fc < 0:
13            b, fb = c, fc
14        else:
15            a, fa = c, fc
16    return (a + b) / 2, max_iter, historico
17
18 # Exemplo:  $x^3 - x - 2 = 0$ 
19 f = lambda x: x**3 - x - 2
20 raiz, iters, hist = bissecao(f, 1, 2, tol=1e-10)
21 print(f"Raiz: {raiz:.15f}, Iteracoes: {iters}")
```

## Vantagens

- Robusto.
- Sempre converge.
- Erro controlado *a priori*.

## Limitações

- Lento (convergência linear).
- Requer intervalo inicial com sinal trocado.

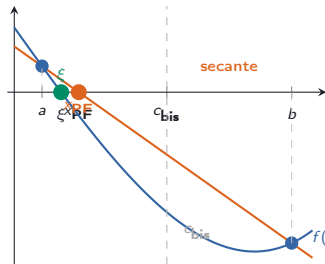
## Ideia: melhorar a bisseção

Na bisseção, o ponto candidato é sempre o **ponto médio** de  $[a, b]$ :

$$x = \frac{a + b}{2}.$$

Mas se  $|f(a)| \ll |f(b)|$ , a raiz provavelmente está muito mais próxima de  $a$  do que de  $b$ .

A **Posição Falsa** (ou *Regula Falsi*) usa uma média **ponderada** pelos valores da função, obtendo aproximações geralmente melhores.



$|f(a)|=1,21 \ll |f(b)|=7,0$ :  $x_{PF} \approx 0,47$  muito mais próximo de  $\xi \approx 0,34$  do que  $c_{bis} = 1,10$ .

### Derivação geométrica

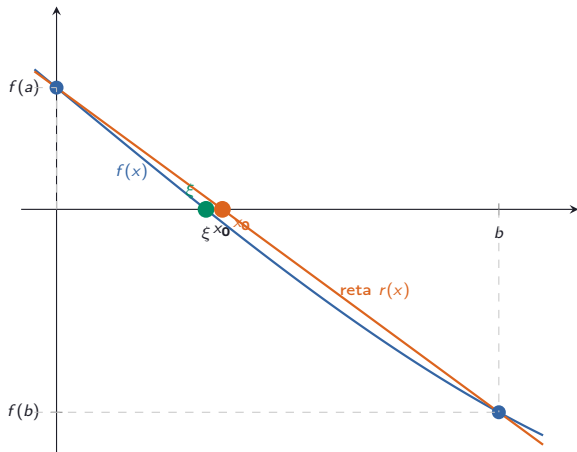
Dados  $a$  e  $b$  com  $f(a)f(b) < 0$ , traçamos a **reta secante** pelos pontos  $(a, f(a))$  e  $(b, f(b))$ . A interseção dessa reta com o eixo  $Ox$  fornece a aproximação:

$$x = \frac{a f(b) - b f(a)}{f(b) - f(a)}$$

que é equivalente à média aritmética **ponderada** de  $a$  e  $b$  com pesos  $|f(b)|$  e  $|f(a)|$ , respectivamente.

## Relação com a biseção

Se  $f(a) = -f(b)$  os pesos são iguais e  $x = \frac{a+b}{2}$  — a biseção é um caso especial da Posição Falsa!



## Como iterar

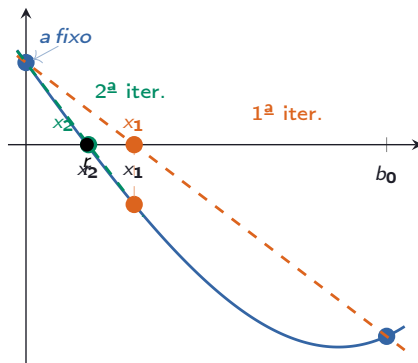
Calculado  $x_k$ , verificamos o sinal de  $f(x_k)$ :

- ▶ Se  $f(a)f(x_k) < 0$ : a raiz está em  $[a, x_k]$ . Fazemos  $b \leftarrow x_k$ .
- ▶ Caso contrário: a raiz está em  $[x_k, b]$ . Fazemos  $a \leftarrow x_k$ .

Repetimos com o novo intervalo.

## Diferença crucial para a bisseção

Na bisseção **os dois** extremos se movem alternadamente. Na Posição Falsa, **um dos extremos pode ficar fixo** por muitas iterações — o que pode tornar a convergência lenta em certos casos.



### Critérios de parada

Parar quando **qualquer** uma das condições abaixo for satisfeita:

$$|b_k - a_k| < \varepsilon_1 \quad \text{ou} \quad |f(x_k)| < \varepsilon_2.$$

Tomar  $r \approx x_k$  como raiz aproximada.

### Theorem (Convergência)

*Se  $f$  é contínua em  $[a, b]$  e  $f(a)f(b) < 0$ , então o Método da Posição Falsa gera uma sequência **convergente** para a raiz  $r \in (a, b)$ .*

### Comparação Bisseção × P. Falsa

|                 | Bisseção        | P. Falsa                        |
|-----------------|-----------------|---------------------------------|
| Ponto candidato | $\frac{a+b}{2}$ | $\frac{af(b)-bf(a)}{f(b)-f(a)}$ |
| Convergência    | Garantida       | Garantida                       |
| Velocidade      | Linear          | Superlinear*                    |
| Exige $f'$      | Não             | Não                             |

\*Em geral mais rápida que a bisseção, mas pode ser lenta se um extremo ficar fixo.

### Ordem de convergência

A ordem do método depende da curvatura de  $f$ : pode variar entre **linear** (extremo fixo) e **superlinear**.

## Algoritmo (Regula Falsi)

**Dados:**  $[a, b]$  com  $f(a)f(b) < 0$ ; precisões  $\varepsilon_1, \varepsilon_2$ ;  $N_{\max}$ .

1. Se  $|f(a)| < \varepsilon_2$ :  $r = a$ . **FIM.**
2. Se  $|f(b)| < \varepsilon_2$ :  $r = b$ . **FIM.**
3.  $M \leftarrow f(a)$ ;  $k \leftarrow 1$ .
4.  $x \leftarrow \frac{a f(b) - b f(a)}{f(b) - f(a)}$
5. Se  $|f(x)| < \varepsilon_2$ :  $r = x$ . **FIM.**
6. Se  $M \cdot f(x) > 0$ :  $a \leftarrow x$ , senão  $b \leftarrow x$ .
7. Se  $|b - a| < \varepsilon_1$ :  $r = x$ . **FIM.**
8.  $k \leftarrow k + 1$ . Se  $k \leq N_{\max}$ : voltar ao passo 4.
9. **Não convergiu. FIM.**

## Observação sobre o passo 6

Comparamos  $f(x)$  com  $M = f(a_0)$ :

- ▶ mesmo sinal  $\Rightarrow$  a raiz está entre  $x$  e  $b$ :  $a \leftarrow x$ .
- ▶ sinais opostos  $\Rightarrow$  a raiz está entre  $a$  e  $x$ :  $b \leftarrow x$ .

Assim garantimos  $f(a)f(b) < 0$  em todos os passos.

## Custo por iteração

1 avaliação de  $f$  por iteração (igual à bissecção).

# Implementação — Posição Falsa

```
1 def posicao_falsa(f, a, b, eps1=1e-8, eps2=1e-8, max_iter=200):
2     fa, fb = f(a), f(b)
3     if fa * fb > 0:
4         raise ValueError("f(a) e f(b) devem ter sinais opostos.")
5     if abs(fa) < eps2: return a, 0, []
6     if abs(fb) < eps2: return b, 0, []
7     historico = []
8     M = fa
9     for k in range(1, max_iter + 1):
10         x = (a*fb - b*fa) / (fb - fa)
11         fx = f(x)
12         historico.append(x)
13         if abs(fx) < eps2 or abs(b - a) < eps1:
14             return x, k, historico
15         if M * fx > 0: # mesmo sinal que f(a) inicial
16             a, fa = x, fx
17         else:
18             b, fb = x, fx
19     return x, max_iter, historico
20
21 # Exemplo: f(x) = x*log10(x) - 1 em [2, 3]
22 import math
23 f = lambda x: x * math.log10(x) - 1
24 raiz, iters, hist = posicao_falsa(f, 2, 3)
25 print(f"Raiz: {raiz:.10f} Iteracoes: {iters}")
```

$$f(x) = x \log_{10}(x) - 1, \quad \text{intervalo } [2, 3], \quad \varepsilon_1 = \varepsilon_2 = 10^{-4}$$

| $k$ | $a_k$  | $b_k$  | $x_k$  | $f(x_k)$    | Novo intervalo   |
|-----|--------|--------|--------|-------------|------------------|
| 1   | 2,0000 | 3,0000 | 2,4715 | -0,0547     | [2,4715, 3,0000] |
| 2   | 2,4715 | 3,0000 | 2,5002 | -0,0003     | [2,5002, 3,0000] |
| 3   | 2,5002 | 3,0000 | 2,5003 | $\approx 0$ | <b>Converge</b>  |

Raiz aproximada:  $r \approx 2,5003$ . Verifique:  $2,5 \log_{10}(2,5) \approx 1,0$  ✓

## Comparação com a Bisseção

A bisseção precisaria de  $k \geq \log_2\left(\frac{1}{10^{-4}}\right) \approx 13$  iterações para o mesmo intervalo e precisão. A Posição Falsa convergiu em **3 iterações**.

## Atenção — extremo fixo

Observe que  $b$  permaneceu em 3,0 em todas as iterações. Isso é típico da Posição Falsa e pode ser problemático em funções com curvatura acentuada.

### Vantagens

- Sempre converge (como a bisseção).
- Em geral mais rápida que a bisseção.
- Não exige derivada.
- Usa informação da **magnitude** de  $f$  — não apenas o sinal.

### Limitações

- Um extremo pode ficar **fixo** (convergência lenta em funções côncavas/convexas muito assimétricas).
- Não há fórmula simples para prever o número de iterações *a priori* (ao contrário da bisseção).
- Curvatura de  $f$  afeta a velocidade.

### Exercícios

1. Execute as 2 primeiras iterações pela Posição Falsa para  $f(x) = x \log_{10}(x) - 1$  em  $[2, 3]$ .
2. Obtenha a raiz aproximada de  $f(x) = x^3 - 9x + 3$  em  $[0, 1]$  com  $\varepsilon_1 = \varepsilon_2 = 2 \times 10^{-3}$ .

## Motivação — Reformular $f(x) = 0$ como $x = g(x)$

### Ideia central

Resolver  $f(x) = 0$  é equivalente a encontrar  $x^*$  tal que

$$x^* = g(x^*) \quad (\text{ponto fixo de } g).$$

A **iteração linear** é simplesmente:

$$x^{(k+1)} = g(x^{(k)}).$$

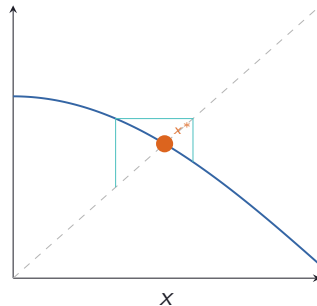


Diagrama de teia de aranha para  $g(x) = \cos(x)$

## Reformular $f(x) = 0$ como $x = g(x)$ – Como?

### Exemplos de reformulação

- ▶  $x^2 - 2 = 0 \Rightarrow g(x) = \frac{2}{x}$  ou  $g(x) = \frac{1}{2}\left(x + \frac{2}{x}\right)$
- ▶  $e^x - 3x = 0 \Rightarrow g(x) = \frac{e^x}{3}$  ou  $g(x) = \ln(3x)$
- ▶  $\cos(x) = x \Rightarrow g(x) = \cos(x)$

### Atenção

Existem **infinitas** funções de iteração para a mesma equação. A escolha de  $g$  determina se o método converge ou diverge!

### Definição

Seja  $g$  uma função contínua em  $[a, b]$ . Dizemos que  $x^*$  é **ponto fixo** de  $g$  se

$$g(x^*) = x^*.$$

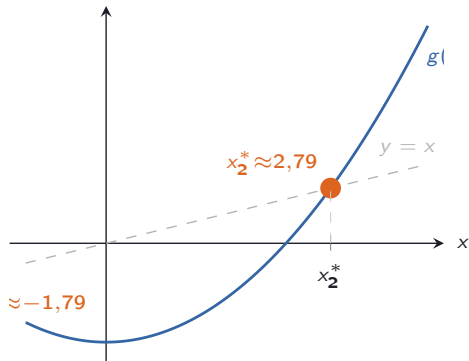
Geometricamente,  $x^*$  é a **abscissa da interseção** das curvas  $y = g(x)$  e  $y = x$ .

## Theorem (Brouwer — Existência)

Se  $g : [a, b] \rightarrow [a, b]$  é contínua, então  $g$  possui  **pelo menos um ponto fixo em  $[a, b]$ .**

## Existência $\Leftrightarrow$ raiz de $f$

Se  $g(x^*) = x^*$  então  $f(x^*) = 0$  para  $f(x) = g(x) - x$ . Os dois problemas são **equivalentes**.



$$g(x^*) = x^* \Rightarrow (x^*)^2 - 5 = x^* \Rightarrow x^* = \frac{1 \pm \sqrt{21}}{2}$$

## Formulação

Dada  $f(x) = 0$  com raiz  $r \in [a, b]$ , reescreva como  $x = g(x)$ , onde  $g : [a, b] \rightarrow [a, b]$  é contínua.

A partir de um **chute inicial**  $x^{(0)}$ , gere:

$$x^{(k+1)} = g(x^{(k)}), \quad k = 0, 1, 2, \dots$$

A sequência  $\{x^{(k)}\}$  converge para  $r$  (sob condições adequadas).

## Cuidado: nem toda $g$ converge!

Para  $x^2 - 2 = 0$ :

►  $g_1(x) = 2/x$ :  $|g'_1(\sqrt{2})| = 1$  — **não converge**

►  $g_2(x) = \frac{1}{2}(x + 2/x)$ :  $g'_2(\sqrt{2}) = 0$  — **quadrática!**

$$1 \rightarrow 1,5 \rightarrow 1,4167 \rightarrow 1,4142 \approx \sqrt{2}$$

## Por que funciona?

Se  $x^{(k)} \rightarrow x^*$  e  $g$  é contínua:

$$x^* = \lim_{k \rightarrow \infty} x^{(k+1)} = \lim_{k \rightarrow \infty} g(x^{(k)}) = g(x^*).$$

Logo  $x^*$  é ponto fixo, i.e.,  $f(x^*) = 0$ .

## Theorem (Mapa de Contração)

Seja  $g : [a, b] \rightarrow [a, b]$  contínua e diferenciável. Se existe  $L < 1$  tal que  $|g'(x)| \leq L$  para todo  $x \in [a, b]$ , então:

1. O ponto fixo  $x^*$  é **único** em  $[a, b]$ .
2. Para qualquer  $x^{(0)} \in [a, b]$ , a iteração  $x^{(k+1)} = g(x^{(k)})$  converge para  $x^*$ .
3. O erro satisfaz:

$$\left| x^{(k)} - x^* \right| \leq \frac{L^k}{1 - L} \left| x^{(1)} - x^{(0)} \right|.$$

## Intuição geométrica

$|g'(x)| < 1$ : a função “achata” distâncias a cada iteração.

- ▶  $|g'(x^*)| > 1$ : diverge.
- ▶  $|g'(x^*)| = 1$ : análise caso a caso.
- ▶  $|g'(x^*)| < 1$ : converge linearmente.
- ▶  $g'(x^*) = 0$ : converge **quadraticamente**.

## Regra prática

Antes de usar  $g$ , **calcule**  $|g'(x)|$  numa vizinhança da raiz estimada. Se  $|g'| > 1$  em toda a vizinhança, troque de formulação.

### Demonstração (esboço)

Seja  $x^*$  ponto fixo. Para dois iterados consecutivos:

$$x^{(k+1)} - x^* = g(x^{(k)}) - g(x^*).$$

Pelo Teorema do Valor Médio, existe  $\xi_k$  entre  $x^{(k)}$  e  $x^*$ :

$$g(x^{(k)}) - g(x^*) = g'(\xi_k)(x^{(k)} - x^*).$$

Tomando valor absoluto:

$$\left| x^{(k+1)} - x^* \right| = |g'(\xi_k)| \left| x^{(k)} - x^* \right| \leq L \left| x^{(k)} - x^* \right|.$$

Aplicando recursivamente:  $\left| x^{(k)} - x^* \right| \leq L^k \left| x^{(0)} - x^* \right| \rightarrow 0.$  ■

### Theorem (Estimativa do Erro — MPF)

*Nas hipóteses do teorema anterior (com  $|g'| \leq L < 1$ ):*

$$\left| x^{(k)} - x^* \right| \leq \frac{L^k}{1 - L} \left| x^{(1)} - x^{(0)} \right|$$

*e também o critério de parada prático:*

$$\left| x^{(k)} - x^* \right| \leq \frac{L}{1 - L} \left| x^{(k)} - x^{(k-1)} \right|.$$

## Critério de parada

Como  $x^*$  é desconhecido, usa-se na prática:

$$\left| x^{(k+1)} - x^{(k)} \right| < \varepsilon \quad \text{ou} \quad \left| f(x^{(k)}) \right| < \varepsilon.$$

## Número de iterações

Para garantir erro  $< \varepsilon$  a partir da estimativa:

$$k \geq \frac{\ln \left( \frac{(1-L)\varepsilon}{|x^{(1)} - x^{(0)}|} \right)}{\ln L}.$$

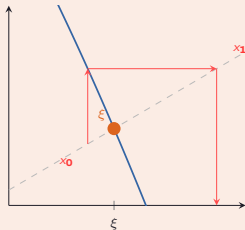
Quatro reescritas de  $x^2 + x - 6 = 0$  como  $x = \varphi(x)$

|             | $\varphi(x)$      | $\varphi'(x)$              | $ \varphi'(2) $              | Resultado |
|-------------|-------------------|----------------------------|------------------------------|-----------|
| $\varphi_1$ | $6 - x^2$         | $-2x$                      | $ -4  = 4$                   | Diverge   |
| $\varphi_2$ | $\sqrt{6 - x}$    | $\frac{-1}{2\sqrt{6 - x}}$ | $\frac{1}{2\sqrt{4}} = 0,25$ | Converge  |
| $\varphi_3$ | $\frac{6}{x} - 1$ | $\frac{-6}{x^2}$           | $\frac{6}{4} = 1,5$          | Diverge   |
| $\varphi_4$ | $\frac{6}{x + 1}$ | $\frac{-6}{(x + 1)^2}$     | $\frac{6}{9} \approx 0,67$   | Converge  |

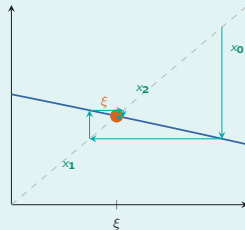
## Observação

Para a **mesma** equação existem infinitas funções de iteração. A convergência depende exclusivamente de  $|\varphi'(\xi)| < 1$ .

$\varphi_1(x) = 6 - x^2$  — Diverge  
( $|\varphi_1'(2)| = 4$ )



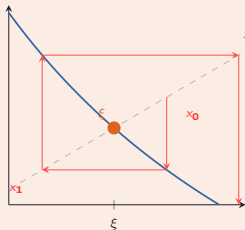
$\varphi_2(x) = \sqrt{6 - x}$  — Converge  
( $|\varphi_2'(2)| = 0,25$ )



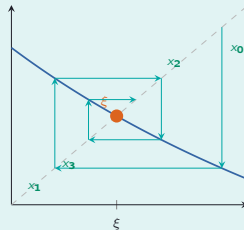
Setas indicam a direção da iteração.  $\varphi_1$ : iterados se afastam de  $\xi$ ;  $\varphi_2$ : espiral convergente para  $\xi = 2$ .

Teia de Aranha —  $\varphi_3(x) = \frac{6}{x} - 1$  e  $\varphi_4(x) = \frac{6}{x+1}$

$\varphi_3(x) = \frac{6}{x} - 1$  — Diverge  
( $|\varphi'_3(2)| = 1,5$ )



$\varphi_4(x) = \frac{6}{x+1}$  — Converge  
( $|\varphi'_4(2)| \approx 0,67$ )



Setas indicam a direção da iteração.  $\varphi_3$ : oscilação **divergente**;  $\varphi_4$ : oscilação **amortecida** convergindo para  $\xi = 2$ .

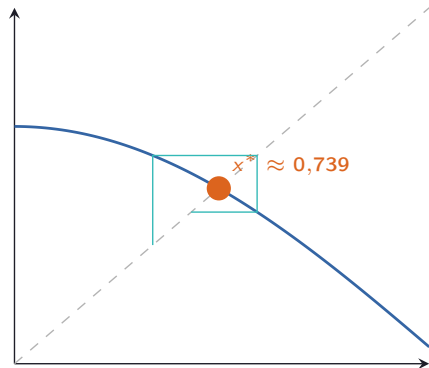
## Exemplo Completo — $\cos(x) = x$

**Equação:**  $f(x) = \cos(x) - x = 0$ .

**Função de iteração:**  $g(x) = \cos(x)$ .

**Verificação:**  $|g'(x)| = |-\sin(x)| < 1$  em  $[0, 1]$ . ✓

| $k$      | $x^{(k)}$ | $ x^{(k)} - x^{(k-1)} $ |
|----------|-----------|-------------------------|
| 0        | 0,5000    | —                       |
| 1        | 0,8776    | 0,3776                  |
| 2        | 0,6390    | 0,2386                  |
| 3        | 0,8027    | 0,1637                  |
| $\vdots$ | $\vdots$  | $\vdots$                |
| 20       | 0,7391    | $< 10^{-4}$             |



$x^* \approx 0,7390851332 \dots$  (**Constante de Dottie**)

$$f(x) = x^3 - x - 1 = 0 \text{ (raiz } x^* \approx 1,3247)$$

Diversas reescritas geram funções de iteração com comportamentos distintos:

|       | $g(x)$                     | $g'(x)$                             | $ g'(x^*) $                | Conclusão       |
|-------|----------------------------|-------------------------------------|----------------------------|-----------------|
| $g_1$ | $x^3 - 1$                  | $3x^2$                              | $3(1,3247)^2 \approx 5,27$ | Diverge         |
| $g_2$ | $\sqrt[3]{x+1}$            | $\frac{1}{3(x+1)^{2/3}}$            | $\approx 0,25$             | Converge        |
| $g_3$ | $\frac{x^3 + 1}{2x^2 - 1}$ | $\frac{x^2(x^2 - 3)}{(2x^2 - 1)^2}$ | $\approx 0,04$             | Converge rápido |

### Como escolher $g$ ?

1. Calcule  $|g'(x^*)|$  (ou estime próximo da raiz).
2. Se  $|g'| < 1$ : método converge.
3. Prefira  $g$  com menor  $|g'(x^*)|$  — converge mais rápido.
4. Newton é o caso ótimo:  $g'(x^*) = 0$ .

### Observação importante

Para a mesma equação existem **infinitas** funções de iteração. A convergência depende exclusivamente da escolha de  $g$ , **não** da equação em si.

## Algoritmo

**Dados:**  $x^{(0)}$ , tolerâncias  $\varepsilon_1$  (erro),  $\varepsilon_2$  (resíduo),  $N_{\max}$ .

1. Para  $k = 0, 1, 2, \dots, N_{\max}$ :
  - 1.1 Calcule  $x^{(k+1)} = g(x^{(k)})$ .
  - 1.2 Se  $|x^{(k+1)} - x^{(k)}| < \varepsilon_1$ , aceite  $r = x^{(k+1)}$ . **FIM.**
  - 1.3 Se  $|f(x^{(k+1)})| < \varepsilon_2$ , aceite  $r = x^{(k+1)}$ . **FIM.**
2. Se  $k = N_{\max}$ : método não convergiu. **AVISO.**

## Propriedades

- Convergência **linear** ( $|g'(x^*)| < 1$ ).
- Não exige derivada.
- Sensível à escolha de  $g$ .
- Precisa de  $g : [a, b] \rightarrow [a, b]$ .

## Vantagens

- Simples de implementar.
- Base teórica para Newton.
- Generalizável a  $\mathbb{R}^n$ .

# Implementação do Ponto Fixo

```
1 def ponto_fixo(g, x0, tol=1e-10, max_iter=100):
2     """Encontra ponto fixo de g via  $x_{k+1} = g(x_k)$ ."""
3     x = x0
4     historico = [x]
5     for k in range(max_iter):
6         x_new = g(x)
7         historico.append(x_new)
8         if abs(x_new - x) < tol:
9             return x_new, k+1, historico
10        x = x_new
11    print("Aviso: convergencia nao atingida.")
12    return x, max_iter, historico
13
14 # Exemplo 1: sqrt(2) via  $g(x) = (x + 2/x)/2$  [quadratica!]
15 import math
16 g_sqrt2 = lambda x: 0.5*(x + 2.0/x)
17 raiz, iters, hist = ponto_fixo(g_sqrt2, 1.0)
18 print(f"sqrt(2) = {raiz:.15f} ({iters} iteracoes)")
19
20 # Exemplo 2: cos(x) = x [linear, ~convergencia lenta]
21 g_cos = lambda x: math.cos(x)
22 raiz2, iters2, _ = ponto_fixo(g_cos, 0.5)
23 print(f"x=cos(x): {raiz2:.10f} ({iters2} iteracoes)")
```

### Theorem (Newton $\equiv$ Ponto Fixo com $g'(x^*) = 0$ )

*Newton para  $f(x) = 0$  é iteração de ponto fixo com:*

$$g(x) = x - \frac{f(x)}{f'(x)}.$$

*Calculando  $g'$ :*

$$g'(x) = 1 - \frac{[f'(x)]^2 - f(x)f''(x)}{[f'(x)]^2} = \frac{f(x)f''(x)}{[f'(x)]^2}.$$

*Em  $x^*$ :  $f(x^*) = 0 \Rightarrow g'(x^*) = 0$ .*

## Conclusão

A convergência **quadrática** de Newton é uma consequência direta de  $g'(x^*) = 0$  no teorema do ponto fixo!

## Resumo — ordens de convergência

| Método     | Ordem           | Exige $f'$ ? |
|------------|-----------------|--------------|
| Bisseção   | 1               | Não          |
| Ponto Fixo | 1 (linear)      | Não          |
| Secante    | $\approx 1,618$ | Não          |
| Newton     | 2               | <b>Sim</b>   |

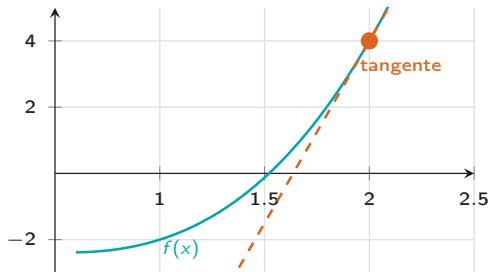
## Newton-Raphson — Derivação Geométrica

Aproximar  $f$  pela tangente em  $x_k$ :

$$L(x) = f(x_k) + f'(x_k)(x - x_k).$$

Raiz de  $L(x) = 0$ :

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$$



### Interpretação

Linearizar localmente e resolver o problema linear — a forma mais simples de obter uma iteração eficiente.

### \* Interpretação

Dobra o número de dígitos corretos a cada iteração!

### Theorem (Convergência Quadrática)

Seja  $f \in C^2$ ,  $r$  raiz simples ( $f(r) = 0$ ,  $f'(r) \neq 0$ ). Com  $x_0$  próximo o suficiente de  $r$ :

$$\lim_{k \rightarrow \infty} \frac{|e_{k+1}|}{|e_k|^2} = \frac{|f''(r)|}{2|f'(r)|}, \quad e_k = x_k - r.$$

### Demonstração — Expansão de Taylor de $f(r)$ em torno de $x_k$

Como  $f \in C^2$ , pelo Teorema de Taylor com resto de Lagrange:

$$f(r) = f(x_k) + f'(x_k)(r - x_k) + \frac{f''(\xi_k)}{2}(r - x_k)^2, \quad \xi_k \text{ entre } x_k \text{ e } r.$$

Como  $f(r) = 0$  e  $e_k = x_k - r$  (logo  $r - x_k = -e_k$ ):

$$0 = f(x_k) - f'(x_k)e_k + \frac{f''(\xi_k)}{2}e_k^2.$$

Dividindo por  $f'(x_k) \neq 0$ :

$$\frac{f(x_k)}{f'(x_k)} = e_k - \frac{f''(\xi_k)}{2f'(x_k)}e_k^2.$$

O erro na iteração seguinte é:

$$\begin{aligned} e_{k+1} &= x_{k+1} - r = \left( x_k - \frac{f(x_k)}{f'(x_k)} \right) - r \\ &= e_k - \frac{f(x_k)}{f'(x_k)} = e_k - \left[ e_k - \frac{f''(\xi_k)}{2f'(x_k)} e_k^2 \right]. \end{aligned}$$

Portanto:

$$e_{k+1} = \frac{f''(\xi_k)}{2f'(x_k)} e_k^2.$$

Quando  $k \rightarrow \infty$ :  $x_k \rightarrow r$ , logo  $\xi_k \rightarrow r$  e  $f'(x_k) \rightarrow f'(r)$ , e assim:

$$\frac{|e_{k+1}|}{|e_k|^2} \rightarrow \frac{|f''(r)|}{2|f'(r)|}.$$



### Theorem

*Se  $r$  tem multiplicidade  $m > 1$ , Newton padrão converge linearmente com fator  $(m - 1)/m$ . O método modificado  $x_{k+1} = x_k - m f(x_k)/f'(x_k)$  restaura a convergência quadrática.*

### Parte 1 — convergência linear

Escreva  $f(x) = (x - r)^m h(x)$  com  $h(r) \neq 0$ . Então:

$$f'(x) = m(x - r)^{m-1}h(x) + (x - r)^m h'(x).$$

A função de iteração de Newton  $g(x) = x - f(x)/f'(x)$  satisfaz:

$$g(x) = x - \frac{(x - r)^m h(x)}{m(x - r)^{m-1}h(x) + (x - r)^m h'(x)} = x - \frac{(x - r)}{m + (x - r)h'(x)/h(x)}.$$

Derivando pela regra do quociente e avaliando em  $r$  (em que  $(x - r) \rightarrow 0$ ):

$$g'(r) = 1 - \frac{1}{m} = \frac{m - 1}{m} \in (0, 1).$$

Logo a convergência é linear com fator  $(m - 1)/m$ .

### Parte 2 — método modificado restaura ordem quadrática

Defina  $\mu(x) = f(x)/f'(x)$ . Das expressões de  $f$  e  $f'$ :

$$\mu(x) = \frac{(x - r)}{m + (x - r) \frac{h'(x)}{h(x)}},$$

portanto

$$\mu'(r) = \frac{1}{m}.$$

A função de iteração do método modificado é  $g_m(x) = x - m\mu(x)$ . Derivando:

$$g'_m(r) = 1 - m\mu'(r) = 1 - m \cdot \frac{1}{m} = 0.$$

Como  $g'_m(r) = 0$ , pelo corolário do Teorema do Ponto Fixo, a convergência é **quadrática**. ■

## ! Casos Problemáticos

1.  $f'(x_k) = 0$ : falha (divisão por zero).
2. Raízes múltiplas: degrada para linear.
3.  $x_0$  longe da raiz: pode divergir.
4. Pontos de inflexão: convergência lenta.
5. Oscilação em certas funções.

## Comparação Prática

Para  $f(x) = x^3 - x - 2$  em  $[1, 2]$ :

| Método   | Iterações | Ordem |
|----------|-----------|-------|
| Bisseção | $\sim 33$ | 1     |
| Secante  | $\sim 8$  | 1.618 |
| Newton   | $\sim 5$  | 2     |

### Exemplo — Raiz Dupla

$f(x) = (x - 2)^2$ , raiz  $x^* = 2$  com  $m = 2$ .

Newton padrão:

$$x_{k+1} = \frac{x_k + 2}{2}$$

Com  $x_0 = 1$ :  $1 \rightarrow 1.5 \rightarrow 1.75 \rightarrow 1.875 \dots$  (lento!)

Newton modificado ( $m = 2$ ):

$$x_{k+1} = x_k - (x_k - 2) = 2$$

Convergência em 1 iteração!

# Implementação de Newton-Raphson

```
1 def newton(f, df, x0, tol=1e-10, max_iter=100):
2     x = x0
3     historico = [x]
4     for k in range(max_iter):
5         fx = f(x)
6         dfx = df(x)
7         if abs(dfx) < 1e-14:
8             raise ValueError(f"Derivada muito pequena em x = {x}")
9         x_new = x - fx / dfx
10        historico.append(x_new)
11        if abs(x_new - x) < tol or abs(fx) < tol:
12            return x_new, k+1, historico
13        x = x_new
14    return x, max_iter, historico
15
16 #  $x^3 - x - 2 = 0$ 
17 f = lambda x: x**3 - x - 2
18 df = lambda x: 3*x**2 - 1
19 raiz, iters, _ = newton(f, df, x0=1.5)
20 print(f"Raiz: {raiz:.15f}, Iteracoes: {iters}")
```

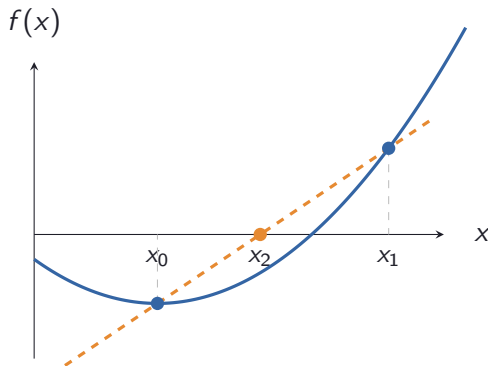
# Método da Secante

**Motivação:**  $f'(x_k)$  pode ser difícil, custosa ou inexistente.

**Ideia:** aproximar derivada por diferença finita:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}$$



## Geometria

Em vez da tangente, usa a reta *secante* pelos pontos  $(x_{k-1}, f(x_{k-1}))$  e  $(x_k, f(x_k))$ .

A partir de duas aproximações  $x_{k-1}$  e  $x_k$ , o ponto  $x_{k+1}$  é obtido como a **abscissa da interseção** do eixo  $Ox$  com a reta secante que passa por  $(x_{k-1}, f(x_{k-1}))$  e  $(x_k, f(x_k))$ .

Substituindo a derivada  $f'(x_k)$  pelo quociente de diferenças na fórmula de Newton:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}$$

obtemos a **função de iteração da secante**:

$$x_{k+1} = \frac{x_{k-1} f(x_k) - x_k f(x_{k-1})}{f(x_k) - f(x_{k-1})}$$

### Observação

São necessárias **duas** aproximações iniciais  $x_0$  e  $x_1$  para iniciar o método. O método pode divergir se  $f(x_k) \approx f(x_{k-1})$ .

### Algoritmo

Seja  $f(x) = 0$ .

1. **Dados iniciais:**  $x_0, x_1$  (aproximações);  $\varepsilon_1, \varepsilon_2$  (precisões).
2. Se  $|f(x_0)| < \varepsilon_2$ , faça  $r = x_0$ . **FIM.**
3. Se  $|f(x_1)| < \varepsilon_2$  ou  $|x_1 - x_0| < \varepsilon_1$ , faça  $r = x_1$ . **FIM.**
4.  $k = 1$
5. 
$$x_2 = x_1 - \frac{f(x_1)}{f(x_1) - f(x_0)} (x_1 - x_0)$$
6. Se  $|x_2 - x_1| < \varepsilon_1$  ou  $|f(x_2)| < \varepsilon_2$ , faça  $r = x_2$ . **FIM.**
7.  $x_0 = x_1$ ;  $x_1 = x_2$ ;  $k = k + 1$ . Volte ao passo 5.

### Critério de parada

$$|f(x_k)| < \varepsilon_1 \quad \text{ou} \quad |x_k - x_{k-1}| < \varepsilon_2.$$

**Exemplo:**  $f(x) = x^2 + x - 6$ ,  $\xi = 2$ ,  $x_0 = 1,5$ ,  $x_1 = 1,8$

$$f(x_0) = f(1,5) = 1,5^2 + 1,5 - 6 = -1,75$$

$$f(x_1) = f(1,8) = 1,8^2 + 1,8 - 6 = -0,96$$

$$x_2 = 1,8 - \frac{-0,96}{-0,96 - (-1,75)} (1,8 - 1,5) = 1,8 - \frac{-0,96}{0,79} \cdot 0,3 \approx 2,165$$

$$f(x_2) = f(2,165) \approx 0,836$$

$$x_3 = 2,165 - \frac{0,836}{0,836 - (-0,96)} (2,165 - 1,8) \approx 1,996$$

## Convergência

O método converge rapidamente para  $\xi = 2$ , sem necessidade de calcular  $f'(x)$ .

### Theorem (Convergência da Secante)

$$|e_{k+1}| \approx C |e_k|^\varphi, \quad \varphi = \frac{1 + \sqrt{5}}{2} \approx 1.618$$

*Razão Áurea!*

### Vantagem Prática

Não requer  $f'$ . Uma avaliação de  $f$  por iteração (vs. duas para Newton).

## Resumo das Ordens de Convergência

| Método   | Ordem           | Avaliações/iter | Derivada?  |
|----------|-----------------|-----------------|------------|
| Bisseção | 1               | 1               | Não        |
| Secante  | $\approx 1.618$ | 1               | Não        |
| Newton   | 2               | 1               | <b>Sim</b> |

Método de Muller (3 pontos): Muller (1956)

### Análise de erros via Taylor

Expanda  $f(x_j)$  em Taylor em torno de  $r$  para  $j = k - 1, k$ :

$$f(x_j) = f'(r) e_j + \frac{1}{2} f''(r) e_j^2 + O(e_j^3).$$

Substituindo na fórmula da secante (os termos  $f'(r)$  se cancelam no quociente):

$$e_{k+1} \approx \frac{f''(r)}{2f'(r)} e_k e_{k-1} =: C \cdot e_k e_{k-1}.$$

### Determinação de $\phi$ — equação funcional

Suponha  $|e_{k+1}| \approx K |e_k|^\phi$ . Então  $|e_k| \approx K |e_{k-1}|^\phi$  e, substituindo:

$$K |e_k|^\phi \approx |C| |e_k| \cdot |e_k|^{1/\phi}.$$

Para que isso valha para todo  $k$ , o expoente deve balancear:  $\phi = 1 + 1/\phi$ , ou seja,

$$\phi^2 - \phi - 1 = 0 \quad \Rightarrow \quad \phi = \frac{1 + \sqrt{5}}{2}.$$



# Implementação da Secante

```
1 def secante(f, x0, x1, tol=1e-10, max_iter=100):
2     historico = [x0, x1]
3     for k in range(max_iter):
4         fx0, fx1 = f(x0), f(x1)
5         if abs(fx1 - fx0) < 1e-14:
6             raise ValueError("Denominador muito pequeno")
7         x_new = x1 - fx1 * (x1 - x0) / (fx1 - fx0)
8         historico.append(x_new)
9         if abs(x_new - x1) < tol or abs(f(x_new)) < tol:
10             return x_new, k+1, historico
11         x0, x1 = x1, x_new
12     return x1, max_iter, historico
13
14 # Comparacao dos tres metodos
15 f = lambda x: x**3 - x - 2
16 df = lambda x: 3*x**2 - 1
17
18 raiz_b, iter_b, _ = bissecao(f, 1, 2)
19 raiz_s, iter_s, _ = secante(f, 1, 2)
20 raiz_n, iter_n, _ = newton(f, df, 1.5)
21 print(f"Bissecao: {iter_b} iters | Secante: {iter_s} iters | Newton: {iter_n} iters")
```

## Theorem (Algoritmo de Horner)

*Para  $p(x) = a_n x^n + \cdots + a_0$ , a recorrência*

$$b_n = a_n, \quad b_i = a_i + x_0 b_{i+1} \quad (i = n-1, \dots, 0)$$

*computa  $p(x_0) = b_0$  com exatamente  $n$  multiplicações e  $n$  adições (custo ótimo).*

## Demonstração por indução finita

**Afirmção:**  $b_i = \sum_{j=i}^n a_j x_0^{j-i}.$

**Base** ( $i = n$ ):  $b_n = a_n = a_n x_0^0.$  a✓

**Passo:** Supondo válido para  $b_{i+1}$ :

$$b_i = a_i + x_0 b_{i+1} = a_i + x_0 \sum_{j=i+1}^n a_j x_0^{j-i-1} = \sum_{j=i}^n a_j x_0^{j-i}. \checkmark$$

Para  $i = 0$ :  $b_0 = \sum_{j=0}^n a_j x_0^j = p(x_0).$

Logo pelo Princípio da Indução Finita segue o resultado. ■

### Horner calcula $p'(x_0)$ sem custo adicional significativo

De  $p(x) = (x - x_0)q(x) + b_0$  com  $q(x) = b_n x^{n-1} + \dots + b_1$ , temos  $p'(x_0) = q(x_0)$ . Aplique Horner a  $q$  em  $x_0$  para obter  $p'(x_0)$  com mais  $n - 1$  operações – essencial para Newton+Horner.

### ! Deflação — Cuidado!

Deflação acumula erros numéricos.

**Solução:** usar deflação apenas para aproximações iniciais, refinar com Newton no polinômio *original*.

### Bônus

Calculando a recorrência **duas vezes** obtemos simultaneamente  $p(x_0)$  e  $p'(x_0)$ , essencial para Newton!

### Exemplo

$p(x) = x^3 - 6x^2 + 11x - 6$  em  $x_0 = 2$ :

$$b_3 = 1$$

$$b_2 = -6 + 2 \cdot 1 = -4$$

$$b_1 = 11 + 2 \cdot (-4) = 3$$

$$b_0 = -6 + 2 \cdot 3 = 0 = p(2) \checkmark$$

### Theorem (Bhaskara Numericamente Estável)

Para  $ax^2 + bx + c = 0$ , calcule:

$$r_1 = \frac{-b - \operatorname{sgn}(b)\sqrt{\Delta}}{2a}, \quad r_2 = \frac{c}{a r_1}, \quad \Delta = b^2 - 4ac.$$

### Justificativa (cancelamento catastrófico)

A fórmula clássica sofre cancelamento quando  $b^2 \gg 4ac$ : por exemplo  $b = -10^6$ ,  $c = 1$ .  $-b - \sqrt{b^2 - 4} \approx 10^6 - 10^6 + \varepsilon$  perde todos os algarismos de  $r_2 \approx 10^{-6}$ . Calculamos  $r_1$  escolhendo o sinal que evita a subtração catastrófica e usamos  $r_1 r_2 = c/a$  (Viète) para obter  $r_2$  com plena precisão.

```
1 def horner_com_derivada(coef, x0):
2     """coef = [a_n, ..., a_0]. Retorna (p(x0), p'(x0))."""
3     p = coef[0]
4     dp = 0.0
5     for c in coef[1:]:
6         dp = dp * x0 + p    # acumula p'
7         p = p * x0 + c      # acumula p
8     return p, dp
9
10 def newton_horner(coef, x0, tol=1e-12, max_iter=50):
11     x = x0
12     for _ in range(max_iter):
13         px, dpx = horner_com_derivada(coef, x)
14         if abs(dpx) < 1e-15: break
15         dx = px / dpx
16         x -= dx
17         if abs(dx) < tol: break
18     return x
19
20 # p(x) = x^3 - 6x^2 + 11x - 6 (raizes 1, 2, 3)
21 coef = [1, -6, 11, -6]
22 print(newton_horner(coef, 1.5)) # -> 1.0
23 print(newton_horner(coef, 2.5)) # -> 3.0
```

Listing 1: Horner com  $p(x_0)$  e  $p'(x_0)$  simultâneos

## Exercícios Propostos

1. Implemente a bisseção e resolva  $f(x) = x^3 - 2x - 5$  em  $[2, 3]$  com  $\varepsilon = 10^{-8}$ . Compare com a previsão teórica de iterações.
2. Mostre que Newton pode divergir se  $f'(x) = 0$  entre  $x_k$  e a raiz. Construa exemplo numérico.
3. Para  $f(x) = (x - 1)^3$ : (a) aplique Newton padrão e observe a convergência linear com fator  $2/3$ ; (b) aplique Newton modificado com  $m = 3$  e confirme a convergência quadrática.
4. Compare bisseção, Newton e secante para  $e^x = 3x$  em  $[0, 1]$ : número de iterações, avaliações de  $f$  e eficiência.
5. Use Horner para avaliar  $p(x) = x^5 - 3x^3 + 2x - 1$  e  $p'(x)$  em  $x = 2$ . Compare tempo com avaliação direta.
6. Encontre todas as raízes de  $p(x) = x^4 - 10x^3 + 35x^2 - 50x + 24$  por deflação sucessiva via Briot-Ruffini. Refine cada raiz em  $p$  original com Newton+Horner.

### Implemente `encontrar_raizes(coef, tol)`

1. Use a **cota de Cauchy** para determinar o intervalo de busca  $[-R, R]$ .
2. Subdivida  $[-R, R]$  em  $N$  subintervalos e identifique trocas de sinal de  $p$ .
3. Em cada subintervalo com troca de sinal, aplique Newton+Horner (ou bisseção como *fallback*).
4. Retorne todas as raízes reais encontradas.

### Teste com:

- $p(x) = x^4 - 10x^3 + 35x^2 - 50x + 24$ ;
- $p(x) = x^5 - 5x^4 + 5x^3 + 5x^2 - 6x$
- polinômios aleatórios de grau 3–6.

## Conceitos Principais

- ▶ Teorema Fundamental da Álgebra.
- ▶ Abel-Ruffini:  $n \geq 5$  sem fórmula.
- ▶ Cota de Cauchy: localização de raízes.
- ▶ Bolzano (TVI): fundamento da bisseção.
- ▶ Ponto fixo: base teórica dos métodos iterativos.

## Métodos Estudados

|            | Ordem      | Exige $f'$ ? |
|------------|------------|--------------|
| Bisseção   | 1          | Não          |
| Ponto Fixo | 1 (linear) | Não          |
| Newton     | 2          | Sim          |
| Secante    | 1,618      | Não          |
| Horner     | —          | —            |

## Pontos de Atenção

- ▶ Escolha de  $g$  determina convergência do MPF.
- ▶ Newton falha com  $f'(x_k) \approx 0$ .
- ▶ Raízes múltiplas degradam Newton (use  $m$  modificado).
- ▶ Deflação acumula erro numérico.
- ▶ Bhaskara tradicional pode ter cancelamento catastrófico.

- Kendall E. Atkinson. *An Introduction to Numerical Analysis*. Wiley, New York, 2 edition, 1989. Cap. 2: Rootfinding for Nonlinear Equations.
- Richard L. Burden and J. Douglas Faires. *Numerical Analysis*. Brooks/Cole, Boston, MA, 9 edition, 2010. Cap. 2: Solutions of Equations in One Variable.
- Gerald Farin. *Curves and Surfaces for CAGD: A Practical Guide*. Morgan Kaufmann, San Francisco, CA, 5 edition, 2002. URL <https://www.sciencedirect.com/book/9781558607378/curves-and-surfaces-for-cagd>.
- David E. Muller. A method for solving algebraic equations using an automatic computer. *Mathematical Tables and Other Aids to Computation*, 10(56):208–215, 1956. doi: 10.2307/2001916. Método de Muller: generalização da secante para três pontos.
- NumPy Developers. NumPy polynomial module. Documentação oficial NumPy, 2024. URL <https://numpy.org/doc/stable/reference/routines.polynomials.html>. Acesso em 2024.

- Katsuhiko Ogata. *Modern Control Engineering*. Prentice Hall, Upper Saddle River, NJ, 5 edition, 2010.
- William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes: The Art of Scientific Computing*. Cambridge University Press, Cambridge, 3 edition, 2007. URL <http://numerical.recipes/>. Cap. 9: Root Finding and Nonlinear Sets of Equations.
- SciPy Developers. SciPy optimization and root finding. Documentação oficial SciPy, 2024. URL <https://docs.scipy.org/doc/scipy/reference/optimize.html>. Acesso em 2024.
- Josef Stoer and Roland Bulirsch. *Introduction to Numerical Analysis*. Springer, New York, 3 edition, 2002. Cap. 5: Finding Zeros and Extrema.
- Wikipedia. Companion matrix. Wikipedia, The Free Encyclopedia, 2024. URL [https://en.wikipedia.org/wiki/Companion\\_matrix](https://en.wikipedia.org/wiki/Companion_matrix). Método para encontrar todas as raízes convertendo para problema de autovalores.